



DESARROLLO DE SOFTWARE PARA PLATAFORMAS MOVILES

JONATHAN LÓPEZ LONDOÑO
JLOPEZL@UAO.EDU.CO
315 926 5443

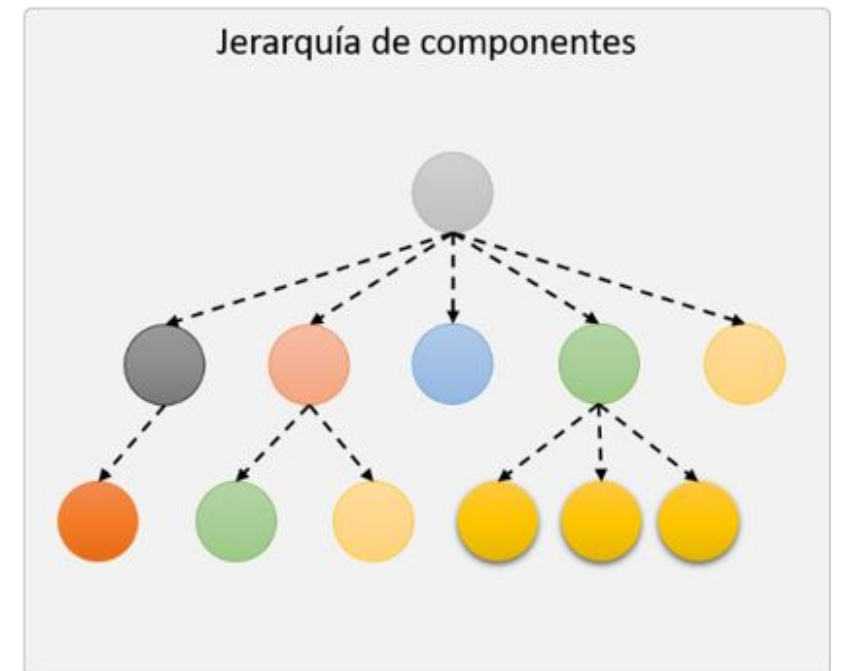
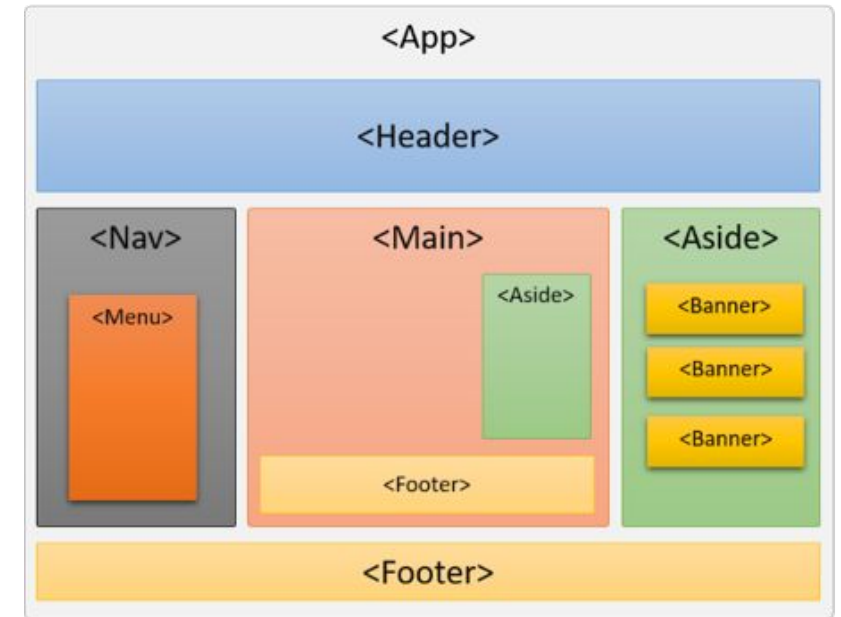
Components

Little piece of reusable encapsuled code which could either contain state or not.

You could create a new React project by using this command:

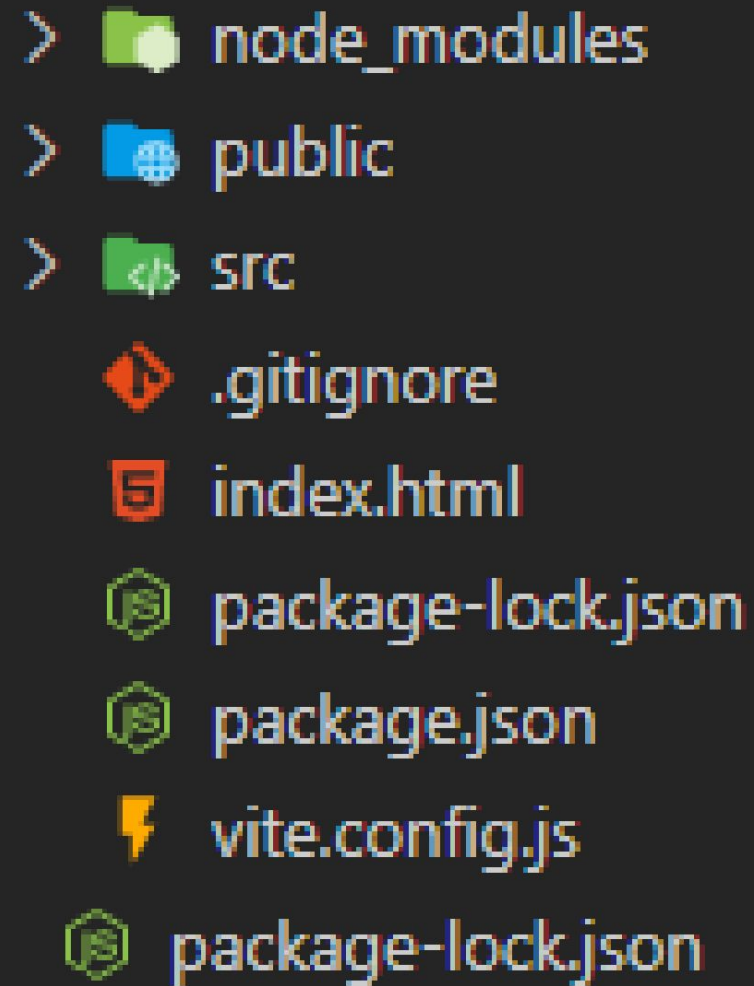
`npm create vite@latest`

Then, follow the instructions in the CMD



Folder Structure

In this structure, our workspace will be **public** and **src**



A screenshot of a file explorer interface with a dark background. It shows a project structure with three folders at the top: 'node_modules' (green icon), 'public' (blue icon with a globe), and 'src' (green icon with a code symbol). Below the folders are several files: '.gitignore' (orange diamond icon), 'index.html' (orange icon with an 'S'), 'package-lock.json' (green icon with 'JS'), 'package.json' (green icon with 'JS'), 'vite.config.js' (yellow lightning bolt icon), and another 'package-lock.json' (green icon with 'JS').

- > node_modules
- > public
- > src
- .gitignore
- index.html
- package-lock.json
- package.json
- vite.config.js
- package-lock.json

Hello World!!

Our first component will be created in **src** folder.

Each component will be a new file, let's call it **HelloWorld.jsx**

To invoke our component we will embed it into a parent.

*Our first parent is **main.jsx***

```
// src/HelloWorld.jsx

function HelloWorld() {
  return (<>
    <h1> Hola Mundo </h1>
  </>);
}

export default HelloWorld;
```

```
// src/main.jsx

import React from 'react';
import ReactDOM from 'react-dom/client';
import HelloWorld from 'HelloWorld';

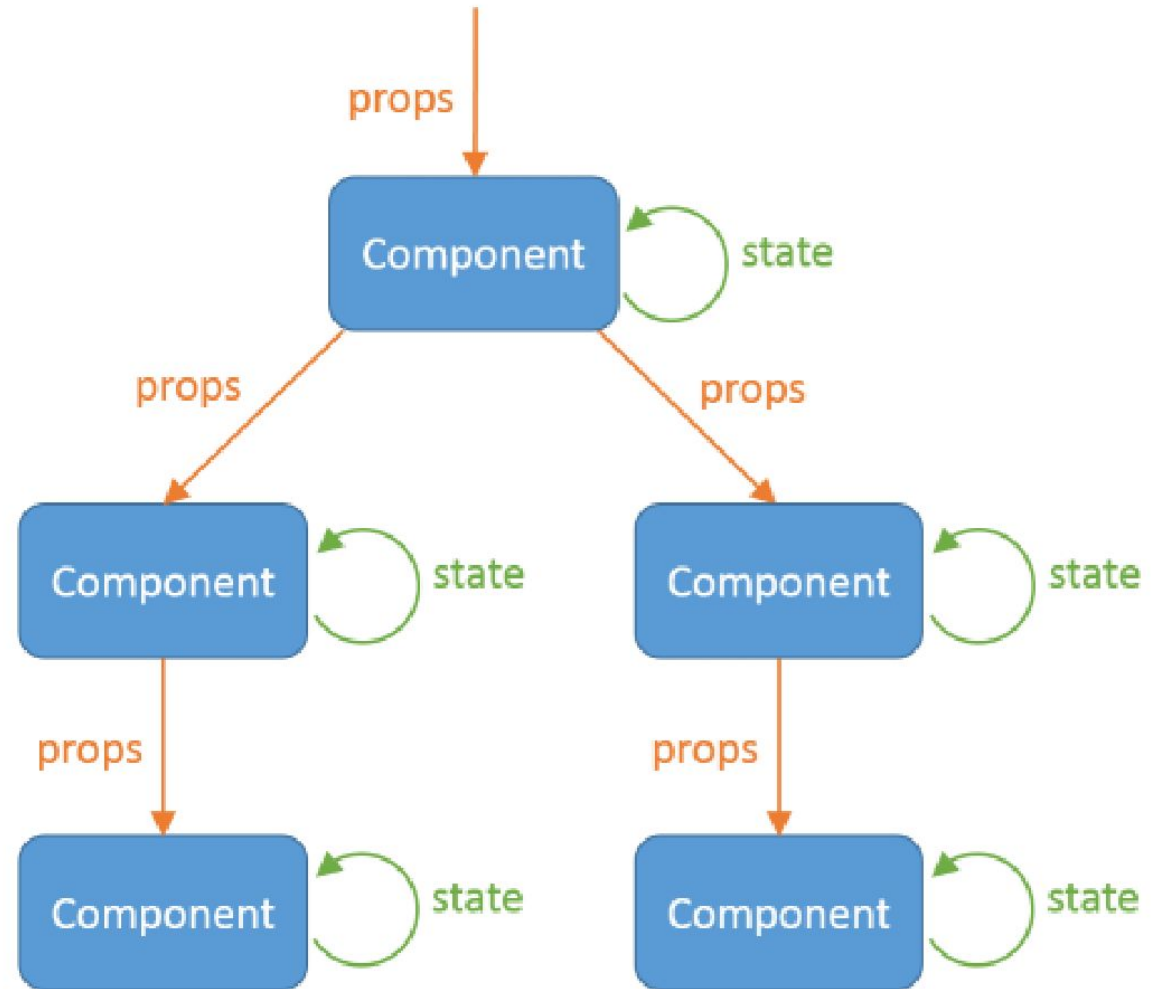
ReactDOM.createRoot(document.getElementById('root'))
.render(
  <React.StrictMode>
    < HelloWorld />
  </React.StrictMode>
)
```

States and Properties

State: data that resides within the component – use them whenever something changes on the screen.

Props: data that a component receives from its parent component.

- They are read-only (cannot be modified)
- They are passed as HTML attributes
- They allow component reuse



Print Variables

To print variables into html we need to use { ... }

We can only print primitive data types directly

- BigInt
- Number
- Strings
- Boolean
- Undefined
- Null
- Symbol

```
// src/ HelloWorld.jsx

const texto = 'First App';

function HelloWorld() {
  return (<>
    <h1> Hola Mundo </h1>
    <h3> Este es mi { texto }</h3>
  </>);
}

export default HelloWorld;
```

Props!

Our first component will be created in **src** folder.

Let's create a new component called **PrintMessage.jsx**

*To invoke our component we will embed it into **main.jsx**.*

```
// src/PrintMessage.jsx
interface Props {
  message: string;
}
function PrintMessage({message}: Props) {
  return (<
    <h3> { message }</h3>
  </>);
}

export default PrintMessage;
```

```
// src/main.jsx

import React from 'react';
import ReactDOM from 'react-dom/client';
import HelloWorld from 'HelloWorld';
import PrintMessage from 'PrintMessage';

ReactDOM.createRoot(document.getElementById('root'))
.render(
  <React.StrictMode>
    < HelloWorld />
    < PrintMessage message="Como te va?" />
    < PrintMessage message="Soy un mensaje!!!" />
  </React.StrictMode>
)
)
```

States

To handle states in react, we need to use a hook (react function) called **useState**.

Use it by importing useState from react and then assigning an initial value to the useState function, like this:

```
import { useState } from 'react';  
const [count, setCount] =  
  useState (initialValue).
```

Don't forget to include your component into some parent

```
// src/ Contador.jsx  
  
import { useState } from 'react';  
  
function Contador() {  
  const [contador, setContador] = useState<number>(0);  
  
  return (  
    <>  
      <p>Contador: {contador}</p>  
      <button onClick={() => setContador(contador + 1)}>  
        Sumar  
      </button>  
    </>  
  );  
}  
  
export default Contador;
```


Print Arrays

To print array into html we need to use the map function

```
{  
  myArray.map(( item, index )=>  
    {...}  
  })  
}
```

```
// src/Arrays.jsx  
  
const myArray = ['First App', 'Second App', 'Third App'] ;  
  
function Arrays() {  
  return (<>  
    <ol>  
      {  
        myArray.map((item, index) => {  
          return <li key={index}> { item } </li>  
        })  
      }  
    </ol>  
  </>);  
}  
  
export default Arrays;
```

Array States

To handle array states in react, we need to use a hook (react function) called **useState**.

```
import { useState } from 'react';  
const [arr, setArr] = useState  
(initialValue).
```

```
const new = 'new value'  
setArr( old => [...old, new] );
```

```
// src/Arreglos.jsx  
  
import { useState } from 'react';  
  
function Arreglos() {  
  const [arreglo, setArreglo] = useState([ ]);  
  
  const addToArray = ( ) => {  
    const nuevo = Date.now();  
    setArreglo( prev => [...prev, nuevo] )  
  }  
  
  return (  
    <>  
      <button onClick={addToArray}> Agregar </button>  
      {  
        arreglo.map((item, index) => {  
          return <li key={index}> { item } </li>  
        })  
      }  
    </>  
  );  
}
```

```
export default Arreglos;
```

Hooks

UseEffect: When using this hook, React must execute something according to the component's lifecycle, ie:

- Calling APIs (fetch)
- Listening for data changes
- Manipulating side effects

The lifecycle will be defined according to the useEffect's array

React executes the effect only once, the array is empty:

```
// src/ EjemploMontaje.jsx

import { useEffect } from 'react';

function EjemploMontaje() {

  useEffect(() => {
    console.log('El componente se montó');
  }, []);

  return <h2>Ejemplo montaje</h2>;
}

export default EjemploMontaje;
```

Hooks

UseEffect.

React executes the effect when the component is mounted, and also when the state changes.

```
// src/ EjemploDependencia.jsx

import { useState, useEffect } from 'react';

function EjemploDependencia() {
  const [nombre, setNombre] = useState("");

  useEffect(() => {
    console.log('El nombre cambió:', nombre);
  }, [nombre]);

  return (
    <>
      <input value={nombre}
        onChange={e => setNombre(e.target.value)}
        placeholder="Escribe tu nombre"
      />
    </>
  );
}

export default EjemploDependencia
```

Hooks

UseEffect

React executes the effect when the component is unmounted, using the return function inside.

```
// src/ EjemploMontaje.jsx

import { useEffect, useState } from 'react';

function EjemploCleanup() {
  const [segundos, setSegundos] = useState(0);

  useEffect(() => {
    const timer = setInterval(() => {
      setSegundos(s => s + 1);
    }, 1000);

    return () => {
      clearInterval(timer);
      console.log('Intervalo limpiado');
    };
  }, []);

  return <p>Segundos: {segundos}</p>;
}

export default EjemploCleanup;
```

Functions

Functions are invoked by prop **events** in HTML

First Way:

```
onClick={ () => function(params) }
```

```
onClick={ (evt) => function(evt,  
params) }
```

Second Way:

```
onClick={ (evt) => function(evt) }
```

```
onClick={function}
```

evt is related to the event itself, will have some attributes like target and this one will have the value from the target

```
// src/ EjemploFunciones.tsx
```

```
import { useEffect, useState } from 'react';
```

```
function EjemploFunciones() {  
  const sumar = (e, valor) => {  
    console.log(e)  
    console.log( 'has presionado un click con ' + valor )  
  }  
}
```

```
return <button onClick={{(evt) => sumar(evt, 4)}}>  
  Sumar  
</button>  
}
```

```
export default EjemploFunciones;
```

HTML – React Events

Event	Use it on
onClick	Buttons, actions
onChange	inputs form
onSubmit	Forms
onBlur	input validations
onFocus	UX
onKeyDown	Keyboard
onmouseenter	Hover
onScroll	Scroll

onClick Examples

```
const sumar = (e) => {  
  console.log(e)  
  console.log( 'has presionado un click' )  
}  
  
return <button onClick={ (evt) => sumar(evt) }>  
  Sumar  
</button>
```

```
const sumar = (e, valor) => {  
  console.log(e)  
  console.log( 'has presionado un click con ' + valor )  
}  
  
return <button onClick={ (evt) => sumar(evt, 4) }>  
  Sumar  
</button>
```


onChange Examples

```
const capturaValor = (e) => {  
  console.log(e)  
  console.log( 'has escrito: ' e.target.value )  
}  
  
return <input  
  placeholder="Escribe tu nombre"  
  onChange={e => capturaValor(e)}  
/>
```


Challenge 01

Build a React app that allows you to list, add, and delete contacts, simulating initial data loading, using the basic concepts that will later be used in Ionic.

The app must:

- Display a loader on startup
- Load an initial contact list
- Allow users to add contacts with name and phone.
- Allow users to delete contacts
- Divide the app into components

